

سوال ۱:

۱. آیا کد کامپایل می شود؟ چرا؟

خیر چون شی s به صورت const تعریف شده است، اما متد getX که روی آن فراخوانی می شود، به صورت const تعریف نشده است.

۲. دقیقاً کدام قانون const نقض شده است؟

فراخوانی یک متد غیر const روی یک شی const مجاز نیست.

۳. کمترین تغییر لازم برای اصلاح کد چیست؟

باید متد getX را به صورت const تعریف کنید

۴. آیا پس از اصلاح، می توان setX را روی شی const فراخوانی کرد؟ چرا؟

خیر چون که setX یک متد غیر const است (زیرا وضعیت شی را تغییر می دهد) و فراخوانی متدهای غیر const بر روی یک شی const (مانند s) همچنان ممنوع است.

سوال ۲:

۱. آیا کد معتبر است؟

خیر

۲. دلیل خطای احتمالی را دقیق توضیح دهید.

متد void Test::f() const یک متد const است. متدهای const تعهد می کنند که هیچ یک از اعضای داده (غیر static) شیئی را که بر روی آن فراخوانی می شوند تغییر ندهند. عبارت x++ دارد سعی میکند که x را تغییر دهد.

۳. اگر بخواهیم x داخل این تابع تغییر کند ولی تابع همچنان const باقی بماند، چه راهی وجود دارد؟

راه حل استفاده از کلمه کلیدی mutable است. اگر عضو داده x به صورت mutable اعلام شود، می توان آن را در داخل متدهای const تغییر داد

۴. آیا این راه حل همیشه توصیه می شود؟

خیر، mutable باید فقط در شرایط خاصی استفاده شود، عمدتاً برای اعضای داده ای که:

- وضعیت منطقی (Logical State) شی را تغییر نمی دهند:

مثلاً برای ذخیره سازی نتایج محاسبات زمان بر (کش کردن)، یا برای پیاده سازی شمارنده های استفاده یا قفل های همگام سازی.

- وضعیت فیزیکی (Physical State) را تغییر می دهند:

مانند یک شمارنده برای ردیابی تعداد دفعات فراخوانی یک متد const

اگر x نمایانگر وضعیت اساسی و منطقی شی باشد، تغییر آن در یک متد const، مفهوم constness را برای کاربر شی نقض می کند.

سوال ۳:

۱. خروجی برنامه چیست؟

3

1

۲. چرا مقدار counter بعد از بلاک داخلی تغییر می کند؟

بلاکی که شامل b, c, Tracker است ، یک Scope ایجاد می کند.

۱. ورود به بلاک: اشیا b و c ساخته می شوند. سازنده Tracker دو بار فراخوانی می شود و counter (که در حال حاضر ۱ است) را به ۳ افزایش می دهد.

۲. خروج از بلاک: پس از cout << Tracker::getCounter() << endl;، اشیا b و c تخریب می شوند.

Tracker~ دو بار فراخوانی می شود و counter (که ۳ است) را به ۱ کاهش می دهد.

۳. پایان برنامه: در نهایت شی a از بین می رود و counter به ۰ می رسد.

۳. نقش static در حفظ وضعیت بین اشیا چیست؟

نقش static این است که عضو داده counter بین تمام اشیا کلاس Tracker به اشتراک گذاشته شود و تنها یک نمونه از آن در حافظه وجود دارد. در ضمن:

- آنها در ابتدای برنامه مقداردهی اولیه می شوند و تا پایان برنامه وجود دارند.
- تمام اشیا Tracker به همان یک counter دسترسی و آن را تغییر می دهند. این قابلیت برای ردیابی وضعیت کلی (مانند تعداد اشیا ایجاد شده یا آمار کلی) بکار می آید.

۴. اگر counter غیر static بود چه اتفاقی می افتاد؟

اگر counter غیر static بود، هر شی Tracker نمونه خصوصی و جداگانه خود را از counter داشت.

فراخوانی Tracker::getCounter() نیز دیگر مجاز نبود، زیرا getCounter نباید به صورت static باشد و روی یک شی فراخوانی شود (مانند a.getCounter()) اگر آن را static نگه می داشتیم، نمی توانست به یک عضو غیر static دسترسی پیدا کند.

سوال ۴:

۱. چرا Car به عنوان friend تعریف شده است؟

Car به عنوان friend کلاس Engine تعریف شده است تا متدهای عضو Car (مانند tune) بتوانند به اعضای خصوصی و محافظت شده کلاس Engine (مانند power) مستقیماً دسترسی داشته باشند.

۲. این طراحی چه اصل OOP را تضعیف می کند؟

این طراحی اصل کیپسوله سازی (Encapsulation) را تضعیف می کند.

۳. چه جایگزین بهتری بدون friend پیشنهاد می دهید؟

جایگزین بهتر این است که Engine یک متد عمومی (Public) برای تغییر power داشته باشد.

۴. در چه شرایطی استفاده از friend قابل توجیه است؟

در شرایط زیر:

- عملیات هایی که به چندین شی از کلاس های مختلف نیاز دارند و هر کلاس برای انجام عملیات به داده های خصوصی کلاس دیگر نیاز دارد.
- هنگامی که دو کلاس ارتباط بسیار نزدیکی دارند و بخشی از یک "مجموعه" هستند، و سرِبار دسترسی از طریق متدهای عمومی ضروری نیست.
- برای الگوهای طراحی خاص (مانند Iterator) که نیاز به دسترسی مستقیم به داده های داخلی دارند.

سوال ۵:

۱. چرا تابع `operator+` به صورت friend نوشته شده است؟

تابع `operator+` یک تابع غیر عضو است که به عنوان friend کلاس `Vector2D` تعریف شده است تا بتواند به اعضای داده خصوصی (`x` و `y`) اشیا `Vector2D` که به عنوان پارامتر به آن ارسال می شوند دسترسی پیدا کند.

۲. آیا می توانست به صورت متد عضو نوشته شود؟

بله، عملگر دوگانه (Binary Operator) مانند `+` می تواند به صورت متد عضو نوشته شود.

۳. تفاوت این دو پیاده سازی در چه سناریوی دیده می شود؟

تفاوت اصلی در سناریوی تبدیل نوع (Type Conversion) برای عمل وند سمت چپ (Left Operand) است.

- متد عضو: فقط زمانی کار می کند که عمل وند سمت چپ شی از نوع `Vector2D` باشد.
 - `v1 + v2` (مجاز)
 - `V1 + 5.0` (مجاز، اگر سازنده تک پارامتری برای `double` وجود داشته باشد)
 - `v1 + 5.0` (نامجاز. کامپایلر نمی داند که چطوری `5.0` را به یک شی `Vector2D` تبدیل کند تا متد `operator+` روی آن فراخوانی شود).
- تابع غیر عضو (`friend`): از آنجا که هر دو عمل وند پارامتر هستند، اگر سازنده مناسبی برای تبدیل `double` به `Vector2D` وجود داشته باشد (مانند `Vector2D(double)`) تبدیل نوع می تواند برای هر دو عمل وند اتفاق بیفتد.
 - `v1 + v2` (مجاز)
 - `V1 + 5.0` (مجاز)
 - `V1 + 5.0` (مجاز)

۴. چرا معمولاً عملگرهای متقارن (`==`, `+`) friend می شوند؟

عملگرهای متقارن (مانند `+`, `-`, `*`, `==`, `!=`) معمولاً به صورت تابع غیر عضو (`Friend` یا `Non-Friend`) پیاده سازی می شوند تا:

۱. تقارن و سازگاری نوع:

امکان پذیر ساختن فراخوانی هایی مانند `V1 + 5.0`، که در آن عمل وند سمت چپ از نوع کلاس نیست

۲. وضوح معنایی:

از نظر منطقی، اضافه کردن دو بردار، یک عمل مستقل است که به هر دو شی بردار به یک اندازه اهمیت می دهد، نه اینکه عمل وندی باشد که توسط بردار اول آغاز شده است. پیاده سازی به صورت تابع غیر عضو این تقارن معنایی را بهتر نشان می دهد.

